POLITECHNIKA MORSKA SZCZECIN

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Architektura systemów komputerowych

Programowanie w asemblerze

Operacje warunkowe

mgr inż. Janusz Magaj

j.magaj@pm.szczecin.pl

Tematy

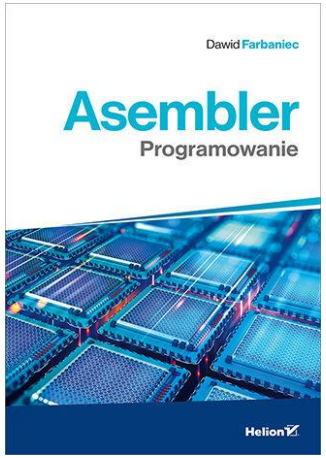
1. Wstęp do architektury komputerów
2. Architektura procesora
3. Reprezentacja i arytmetyka liczb całkowitych w procesorze
4. Reprezentacja i arytmetyka liczb zmiennoprzecinkowych w procesorze
5. **Programowanie w asemblerze**

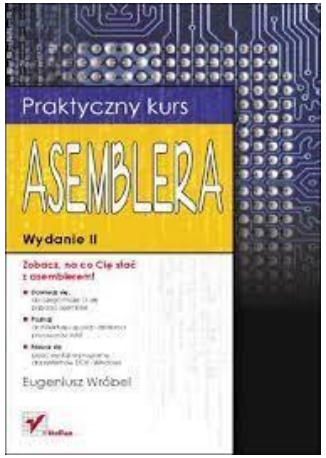
    1. Wstęp - asembler
    2. Operacje przypisania, operacje arytmetyczne
    3. **Operacje warunkowe**
    4. Pętle programowe
    5. Tablice, wskaźniki
    6. Stos, Funkcje, wywołanie funkcji
    7. Operacje zmiennoprzecinkowe

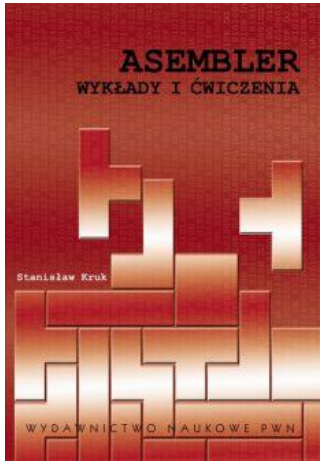
6. Budowa komputera PC

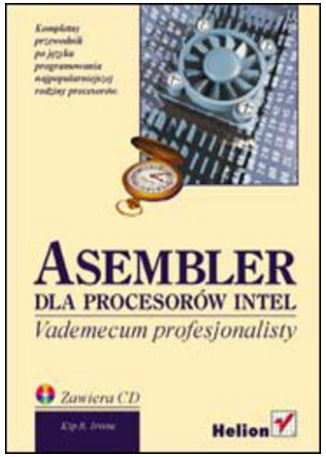
Literatura

- Farbaniec D., Asembler Programowanie, Helion, 2019
- Wróbel E., Praktyczny kurs ASEMBLERA, Helion, 2011
- Kruk S., Asembler Wykłady i ćwiczenia, WN PWN, 2009
- Pirogow V., Asembler. Podręcznik programisty, Helion, 2005
- Irvine K.R., Asembler dla procesorów Intel. Vademecum profesjonalisty, Helion, 2003
- Materiały firmowe – INTEL, AMD, Apple, Qualcomm

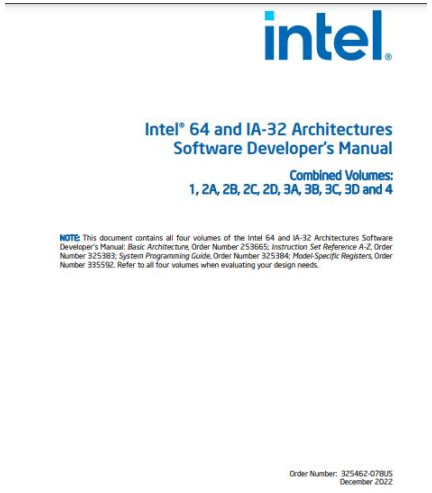


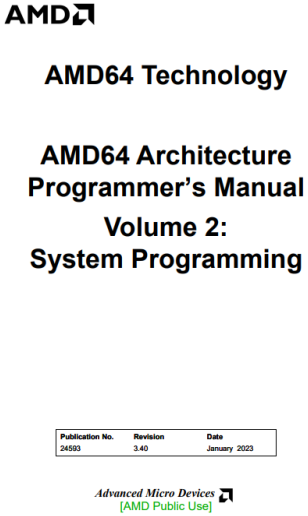


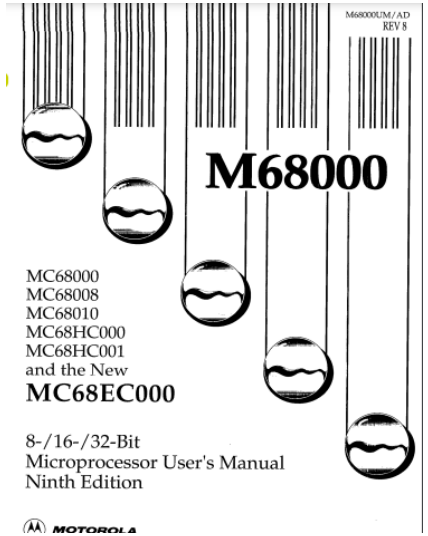


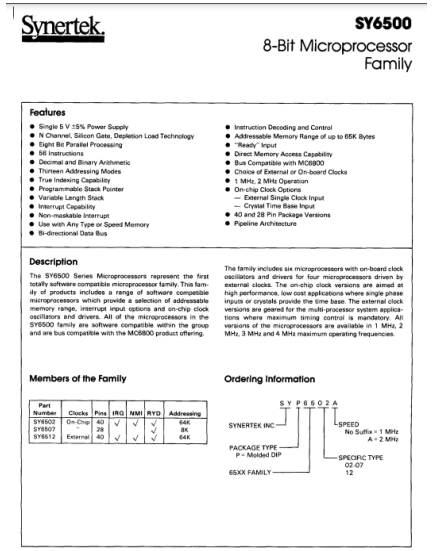


Materiały firmowe:









Microsoft | Learn Dokumentacja Szkolenie Certyfikaty Q&A Przykłady kodu Oceny Program Zdarzenia Wyszukaj Zaloguj

Wersja

Visual Studio 2022

Filtruj według tytułu

- Funkcje wewnętrzne kompilatora i język asemblera
- Funkcje wewnętrzne kompilatora
- Wbudowany asembler Microsoft C/C++ x86
- Asembler ARM i ARM64 — dokumentacja
- Asembler x86 i x64 — dokumentacja
  - Microsoft Macro Assembler (MASM) — dokumentacja
  - MASM dla wersji x64 (ml64.exe)
  - Format instrukcji MASM
  - Informacje w wierszu polecenia programu ML i ML64
- Dokumentacja dyrektyw (MASM)

... / Asembler x86 i x64 / Asembler x86 i x64 — dokumentacja /

MASM dla wersji x64 (ml64.exe)

Artykuł • 10.01.2023 • Czas czytania: 2 min • Współautorzy: 7

Opinia

Program Visual Studio zawiera zarówno 32-bitowe, jak i 64-bitowe wersje oprogramowania MASM (Microsoft Macro Assembler) do docelowego kodu x64. Nazwany ml64.exe to asemblera, który akceptuje język asemblera x64. Narzędzia wiersza polecenia MASM są instalowane podczas wybierania obciążenia języka C++ podczas instalacji programu Visual Studio. Narzędzia MASM nie są dostępne jako oddzielny plik do pobrania. Aby uzyskać instrukcje dotyczące pobierania i instalowania kopii programu Visual Studio, zobacz Instalowanie programu Visual Studio. Jeśli potrzebujesz tylko narzędzi wiersza polecenia, a nie pełnego środowiska IDE, pobierz narzędzia Build Tools for Visual Studio.

Aby użyć ml64.exe w wierszu polecenia, uruchom wiersz polecenia dla deweloperów dla elementów docelowych x64. Wiersz polecenia dla deweloperów ustawia wymaganą ścieżkę i inne zmienne środowiskowe. Aby uzyskać informacje na temat uruchamiania wiersza polecenia dla deweloperów, zobacz Kompilowanie kodu C/C++ w wierszu polecenia.

W tym artykule

- Dodawanie pliku języka asemblera do projektu visual Studio C++
- dyrektywy specyficzne dla ml64
- Tryb 32-bitowego adresu (zastępowanie rozmiaru adresu)
- Zobacz też

Instrukcja warunkowa

Instrukcja warunkowa – element języka programowania, który pozwala na wykonanie różnych instrukcji w zależności od tego czy zdefiniowane przez programistę wyrażenie logiczne jest prawdziwe, czy fałszywe.

$ax^2 + bx + c \quad a \neq 0$

Najpierw liczę deltę $\Delta = b^2 - 4ac$

Pierwiastki równania kwadratowego liczę w zależności od znaku delty:

$\Delta > 0$ *dwa pierwiastki* $x_1 = \frac{-b - \sqrt{\Delta}}{2a}$ $\quad x_1 = \frac{-b + \sqrt{\Delta}}{2a}$

$\Delta = 0$ *jeden pierwiastek* $x_1 = \frac{-b}{2a}$

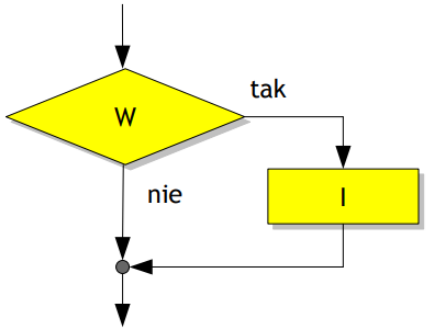
$\Delta < 0$ *brak pierwiastków*

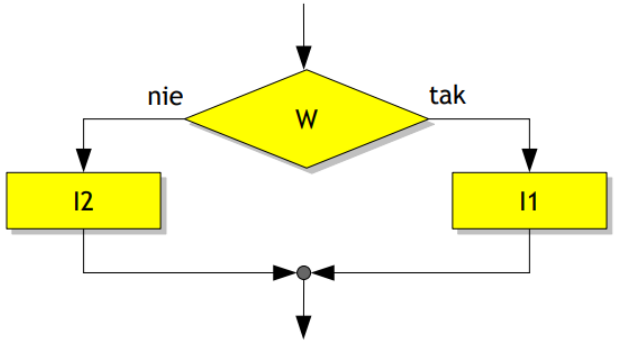
Instrukcja warunkowa - rodzaje

If-Then

Jest spotykany w większości języków programowania i umożliwia warunkowe wykonanie określonego bloku kodu, a jeśli warunek nie jest spełniony – alternatywnego bloku.

```
if warunek then
    właściwy blok kodu
else
    alternatywny blok kodu
end if
```





Instrukcja warunkowa - rodzaje

If-Then

else-if

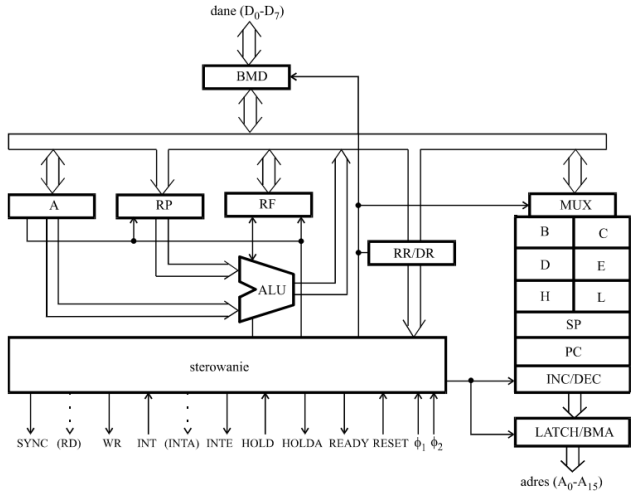
```
if warunek 1 then
    pierwszy blok kodu
else-if warunek 2 then
    drugi blok kodu
else-if warunek 3 then
    trzeci blok kodu
else
    alternatywny blok kodu
end if
```

Wiele języków programowania umożliwia zdefiniowanie więcej niż jednego warunku do sprawdzenia przy pomocy opcjonalnego bloku ***else-if***

Instrukcja warunkowa - warunek

W asemblerze, wynik porównania jest przechowywany w rejestrze **FLAGS** dla późniejszego użycia.

- 80x86 dostarcza instrukcji **CMP** dla wykonania porównania.
- Rejestr **FLAGS** jest ustawiany w oparciu o różnicę dwóch operandów instrukcji **CMP**. Operandy są odejmowane a rejestr **FLAGS** jest ustawiany w oparciu o wynik, ale wynik nie jest nigdzie przechowywany. Jeśli chcemy użyć wyniku stosujemy instrukcję **SUB** zamiast **CMP**



Instrukcja warunkowa - warunek

W asemblerze, wynik porównania jest przechowywany w rejestrze **FLAGS** dla późniejszego użycia.

8080

S	Z	0	AC	0	P	1	CY

0 **CY (carry)** – przeniesienie – bit jest ustawiany, gdy na najstarszym bicie wyniku operacji wystąpiło przeniesienie lub pożyczka,
1 Stała wartość 1,
2 **P (parity)** – parzystość – bit jest ustawiany, gdy bitowy zapis wyniku posiada parzystą ilość jedynek,
3 Stała wartość 0,
4 **AC (auxillary carry)** – przeniesienie połówkowe - bit jest ustawiany, gdy wystąpi pożyczka lub przeniesienie między bitem trzecim i czwartym wyniku,
5 Stała wartość 0,
6 **Z (zero)** – bit jest ustawiany, gdy wynik jest równy 0,
7 **S (sign)** – znak – bit jest ustawiany gdy najstarszy bit wyniku ma wartość 1.

Flagi - przykłady

0 **CY (carry)** – przeniesienie – bit jest ustawiany, gdy na najstarszym bicie wyniku operacji wystąpiło przeniesienie lub pożyczka,

1010 1110
+0111 0100

10010 0010
CY=1

```
.486
.model flat, stdcall
.stack 4096
.data

.code
main proc
        xor eax,eax;
        mov al, 174;
        add al, 116;
        nop;
main    ENDP
END         main
```

Rejestry

```
EAX = 000000AE EBX = 0093B000 ECX = 00F61005 EDX = 00F61005
   EBP = 00AFFA6C EFL = 00000246

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000022 EBX = 0093B000 ECX = 00F61005 EDX = 00F6100
   EBP = 00AFFA6C EFL = 00000217

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 1 PE = 1 CY = 1
```

2 **P (parity)** – parzystość – bit jest ustawiany, gdy bitowy zapis wyniku posiada parzystą ilość jedynek,

- CY - CF (Carry Flag)
- PE - PF (Parity Flag)
- AC - AF (Auxiliary Carry Flag)
- ZR - ZF (Zero Flag)
- PL - SF (Sign Flag)
- EI - IF (Interrupt Enable Flag)
- UP - DF (Direction Flag)
- OV - OF (Overflow Flag)

Flagi - przykłady

4 **AC (auxillary carry)** – przeniesienie połówkowe - bit jest ustawiany, gdy wystąpi pożyczka lub przeniesienie między bitem trzecim i czwartym wyniku,

0010 1110
+0110 0100

1001 0010
AF=**1**

```
main proc
        xor eax,eax;
        mov al, 46;
        add al, 52;
        nop;   ≤ 2 ms upłynęło
main    ENDP
```

Rejestry

```
EAX = 0000002E EBX = 005F8000 ECX = 007F1005 EDX = 007F1005
   EBP = 0075F9DC EFL = 00000246

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000062 EBX = 005F8000 ECX = 007F1005 EDX = 007F1005
   EBP = 0075F9DC EFL = 00000212

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 1 PE = 0 CY = 0
```

- CY - CF (Carry Flag)
- PE - PF (Parity Flag)
- AC - AF (Auxiliary Carry Flag)
- ZR - ZF (Zero Flag)
- PL - SF (Sign Flag)
- EI - IF (Interrupt Enable Flag)
- UP - DF (Direction Flag)
- OV - OF (Overflow Flag)

Flagi - przykłady

6 **Z (zero)** – bit jest ustawiany, gdy wynik jest równy 0,

1111 1111
+0000 0001

0000 0000
ZF=**1**

```
.code
main proc
        xor eax,eax;
        mov al, 255;
        add al, 1;
        nop;
main    ENDP
END         main
```

Rejestry

```
EAX = 000000FF EBX = 0084A000 ECX = 00E11005 EDX = 00E11005
   EBP = 007EF928 EFL = 00000246

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000000 EBX = 00578000 ECX = 002A1005 EDX = 002A1005
   EBP = 003EF8C0 EFL = 00000257

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 1 PE = 1 CY = 1
```

- CY - CF (Carry Flag)
- PE - PF (Parity Flag)
- AC - AF (Auxiliary Carry Flag)
- ZR - ZF (Zero Flag)
- PL - SF (Sign Flag)
- EI - IF (Interrupt Enable Flag)
- UP - DF (Direction Flag)
- OV - OF (Overflow Flag)

```
7  main proc
8          xor eax,eax;
9          mov al, 255;
0          add al, 2;
```

Rejestry

```
EAX = 00000001 EBX = 0084A000 ECX = 00E11005 EDX = 00E11005 ES
   EBP = 007EF928 EFL = 00000213

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 1 PE = 0 CY = 1
```

Flagi - przykłady

7 **S (sign)** – znak – bit jest ustawiany gdy najstarszy bit wyniku ma wartość 1.

```
 0110 0000
+0100 0001

 1010 0001
SF=1
```

```
main proc
        xor eax,eax;
        mov al, 96;
        add al, 65;
        nop;   ≤ 3 ms upłynęło
main    ENDP
```

Rejestry

```
EAX = 00000060 EBX = 00E6B000 ECX = 00411005 EDX = 00411005
   EBP = 010FFAC0 EFL = 00000246

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 000000A1 EBX = 00E6B000 ECX = 00411005 EDX = 00411005
   EBP = 010FFAC0 EFL = 00000A82

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 1 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 0 CY = 0
```

- CY - CF (Carry Flag)
- PE - PF (Parity Flag)
- AC - AF (Auxiliary Carry Flag)
- ZR - ZF (Zero Flag)
- PL - SF (Sign Flag)
- EI - IF (Interrupt Enable Flag)
- UP - DF (Direction Flag)
- OV - OF (Overflow Flag)

Rejestr flag - FLAGS

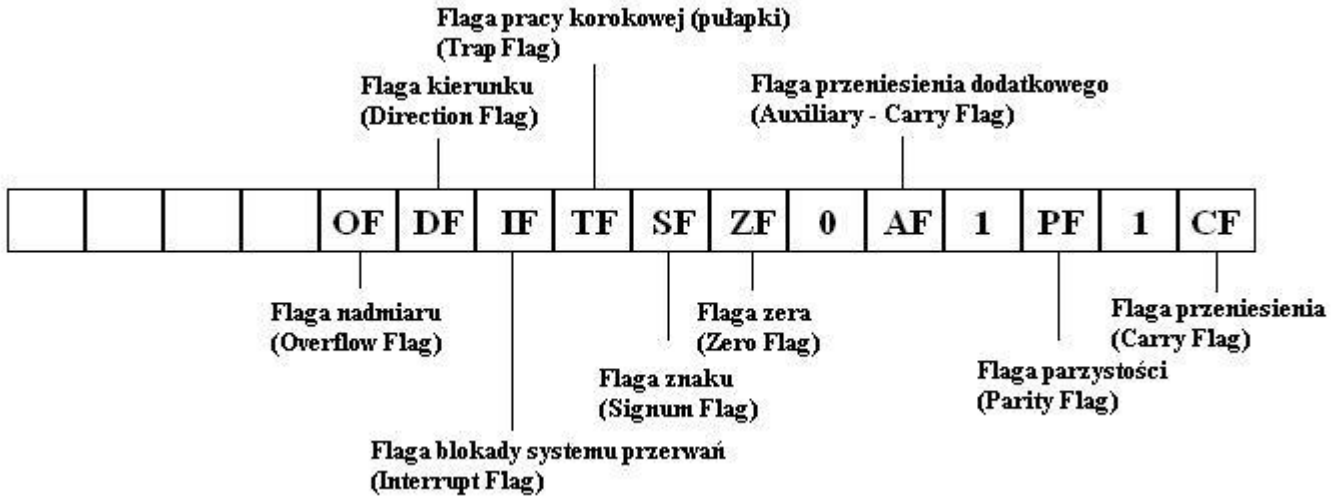
FLAGS – rejestr procesorów zgodnych z architekturą **x86** zaprojektowaną przez **Intela** w latach 70. XX wieku i modernizowaną do tej pory, na której opiera się zdecydowana większość współczesnych komputerów osobistych.

Nazwa FLAGS dotyczy zwykle 16-bitowego rejestru stanu procesorów do 80286 włącznie.

Rejestr stanu architektury x86 firmy Intel

Nr bitu	Skrót	Opis	Kategoria[a]
		FLAGS	
0	CF	flaga przeniesienia (*carry flag*)	S
1	1	*zarezerwowane*	
2	PF	flaga parzystości (*parity flag*)	S
3	0	*zarezerwowane*	
4	AF	flaga wyrównania (*adjust flag*)	S
5	0	*zarezerwowane*	
6	ZF	flaga zera (*zero flag*)	S
7	SF	flaga znaku (*sign flag*)	S
8	TF	flaga pracy krokowej (*trap flag / single step flag*)	X
9	IF	flaga przerwań (*interrupt enable flag*)	X
10	DF	flaga kierunku (*direction flag*)	C
11	OF	flaga przepełnienia (*overflow flag*)	S
12, 13	IOPL	poziom uprzywilejowania we/wy (*I/O privilege level*, 286+)	X
14	NT	flaga zadania zagnieżdżonego (*nested task flag*; 286+)	X
15	0	*zarezerwowane*	

8086



Rejestr flag - FLAGS

- W architekturze IA-32 (od 80386 do najnowszych 32-bitowych procesorów firmy Intel) rejestr ten jest 32-bitowy i nosi nazwę **EFLAGS**.
- W procesorach o architekturze 64-bitowej, zgodnej z x86 (x86-64, EM64T) rejestr stanu ma długość 64 bitów, przy czym opisuje się go symbolem **RFLAGS**.

EFLAGS			
16	RF	flaga wznowienia (*resume flag*, 386+)	X
17	VM	flaga trybu wirtualnego 8086 (*virtual 8086 mode flag*; 386+)	X
18	AC	sprawdzenie wyrównania (*alignment check*; 486SX+)	X
19	VIF	flaga przerwania wirtualnego (*virtual interrupt flag*; Pentium+)	X
20	VIP	oczekujące przerwanie wirtualne (*virtual interrupt pending*; Pentium+)	X
21	ID	identyfikacja (*identification*; Pentium+)	X
22	0	*zarezerwowane*	
23	0	*zarezerwowane*	
24	0	*zarezerwowane*	
25	0	*zarezerwowane*	
26	0	*zarezerwowane*	
27	0	*zarezerwowane*	
28	0	*zarezerwowane*	
29	0	*zarezerwowane*	
30	0	*zarezerwowane*	
31	0	*zarezerwowane*	
RFLAGS			
32-63	0	*zarezerwowane*	

Poszczególne rodzaje rejestrów są ze sobą zgodne

Rejestr flag - FLAGS

- Wykonanie niektórych instrukcji może zmienić wartości w rejestrze flag

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
AAA	—	—	—	TM	—	M					
AAD	—	M	M	—	M	—					
AAM	—	M	M	—	M	—					
AAS	—	—	—	TM	—	M					
ADC	M	M	M	M	M	TM					
ADD	M	M	M	M	M	M					
AND	0	M	M	—	M	0					
ARPL			M								
BOUND											
BSF/BSR	—	—	M	—	—	—					
BSWAP											
BT/BTS/BTR/BTC	—	—	—	—	—	M					
CALL											
CBW											
CLC						0					
CLD									0		

T	Instruction tests flag.
M	Instruction modifies flag (either sets or resets depending on operands).
0	Instruction resets flag.
1	Instruction sets flag.
—	Instruction's effect on flag is undefined.
R	Instruction restores prior value of flag.
Blank	Instruction does not affect flag.

ADD	Integer add
ADC	Add with carry

Rejestr flag - FLAGS

- Wykonanie niektórych instrukcji może zmienić wartości w rejestrze flag

INSTRUCTION SET REFERENCE

intel

ADD—Add

Opcode	Instruction	Description
04 *ib*	ADD AL,*imm8*	Add *imm8* to AL
05 *iw*	ADD AX,*imm16*	Add *imm16* to AX
05 *id*	ADD EAX,*imm32*	Add *imm32* to EAX
80 /0 *ib*	ADD *r/m8,imm8*	Add *imm8* to *r/m8*
81 /0 *iw*	ADD *r/m16,imm16*	Add *imm16* to *r/m16*
81 /0 *id*	ADD *r/m32,imm32*	Add *imm32* to *r/m32*
83 /0 *ib*	ADD *r/m16,imm8*	Add sign-extended *imm8* to *r/m16*
83 /0 *ib*	ADD *r/m32,imm8*	Add sign-extended *imm8* to *r/m32*
00 /*r*	ADD *r/m8,r8*	Add *r8* to *r/m8*
01 /*r*	ADD *r/m16,r16*	Add *r16* to *r/m16*
01 /*r*	ADD *r/m32,r32*	Add r32 to *r/m32*
02 /*r*	ADD *r8,r/m8*	Add *r/m8* to *r8*
03 /*r*	ADD *r16,r/m16*	Add *r/m16* to *r16*
03 /*r*	ADD *r32,r/m32*	Add *r/m32* to *r32*

Description

Adds the first operand (destination operand) and the second operand (source operand) and stores the result in the destination operand. The destination operand can be a register or a memory location; the source operand can be an immediate, a register, or a memory location. (However, two memory operands cannot be used in one instruction.) When an immediate value is used as an operand, it is sign-extended to the length of the destination operand format.

The ADD instruction performs integer addition. It evaluates the result for both signed and unsigned integer operands and sets the OF and CF flags to indicate a carry (overflow) in the signed or unsigned result, respectively. The SF flag indicates the sign of the signed result.

This instruction can be used with a LOCK prefix to allow the instruction to be executed atomically.

Operation

DEST ← DEST + SRC;

Flags Affected

The OF, SF, ZF, AF, CF, and PF flags are set according to the result.

ADD	Integer add
ADC	Add with carry

Instrukcja dodawania całkowitego

Operation

DEST ← DEST + SRC;

Flags Affected

The OF, SF, ZF, AF, CF, and PF flags are set according to the result.

Instrukcja skoku

- Wykorzystując ustawienia flag można wykonać **instrukcje skoku**

intel

PROGRAMMING WITH THE GENERAL-PURPOSE INSTRUCTIONS

Table 7-4. Conditional Jump Instructions

Instruction Mnemonic	Condition (Flag States)	Description
Unsigned Conditional Jumps		
JA/JNBE	(CF or ZF)=0	Above/not below or equal
JAE/JNB	CF=0	Above or equal/not below
JB/JNAE	CF=1	Below/not above or equal
JBE/JNA	(CF or ZF)=1	Below or equal/not above
JC	CF=1	Carry
JE/JZ	ZF=1	Equal/zero
JNC	CF=0	Not carry
JNE/JNZ	ZF=0	Not equal/not zero
JNP/JPO	PF=0	Not parity/parity odd
JP/JPE	PF=1	Parity/parity even
JCXZ	CX=0	Register CX is zero
JECXZ	ECX=0	Register ECX is zero
Signed Conditional Jumps		
JG/JNLE	((SF xor OF) or ZF) =0	Greater/not less or equal
JGE/JNL	(SF xor OF)=0	Greater or equal/not less
JL/JNGE	(SF xor OF)=1	Less/not greater or equal
JLE/JNG	((SF xor OF) or ZF)=1	Less or equal/not greater
JNO	OF=0	Not overflow
JNS	SF=0	Not sign (non-negative)
JO	OF=1	Overflow
JS	SF=1	Sign (negative)

jc etykieta

skok jak wystąpiło przeniesienie

Instrukcja warunkowa - warunek

Przykład:

```
        cmp eax, ebx
        jle else_
        mov ecx, ebx
        jmp koniec
else_:  mov ecx, eax
        sub ecx, ebx
koniec:
```

```
if( eax>ebx )
  ecx = ebx;
else
  ecx = eax - ebx;
```

Przykład 100+100 = ?

Rejestry

```
EAX = 00AFFB64 EBX = 0084F000 ECX = 00F71005 EDX = 00F71005 E

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

85 %

Pamięć 1 | Rejestry

Dezasemblacja | zad_if1.asm

```
1   .486
2   .model flat, stdcall
3   .stack 4096
4
5   ExitProcess PROTO, dwExitCode:DWORD
6
7   ;extern ExitProcess@4 :proc
8   extern MessageBoxA@16 :proc
9
10  .data
11      wynik BYTE ?
12  .code
13
14  main proc
15
16          xor al,al;
17          mov al, 100;
18          add al, 100;   ≤3 ms upłynęło
19          mov wynik, al;
20          nop;
```

Rejestry

```
EAX = 00AFFBC8 EBX = 0084F000 ECX = 00F71005 EDX = 00F71005 E

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 1 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 0 CY = 0
```

85 %

Pamięć 1 | Rejestry

Dezasemblacja | zad_if1.asm

```
1   .486
2   .model flat, stdcall
3   .stack 4096
4
5   ExitProcess PROTO, dwExitCode:DWORD
6
7   ;extern ExitProcess@4 :proc
8   extern MessageBoxA@16 :proc
9
10  .data
11      wynik BYTE ?
12  .code
13
14  main proc
15
16          xor al,al;
17          mov al, 100;
18          add al, 100;
19          mov wynik, al;   ≤2 ms up
20          nop;
21
```

$AL = C8_{16} = 200_{10}$

Dane wyjściowe

Pokaż dane wyjściowe z: Debugowanie

```
„Project1.exe” (Win32): załadowano „C:\Windows\SysWOW64\imm32.dll”.
Wątek 0x7598 zakończył działanie z kodem 200 (0xc8).
Wątek 0x6c0c zakończył działanie z kodem 0 (0x0).
Wątek 0x8c64 zakończył działanie z kodem 200 (0xc8).
Program „[40560] Project1.exe” zakończył działanie z kodem 200 (0xc8).
```

Przykład 200+100 = ?

Rejestry

```
EAX = 00CFFAC8 EBX = 00A35000 ECX = 01001005 EDX = 01001005 E

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

85 %

Pamięć 1 | Rejestry

Dezasemblacja | zad_if1.asm

```
1   .486
2   .model flat, stdcall
3   .stack 4096
4
5   ExitProcess PROTO, dwExitCode:DWORD
6
7   ;extern ExitProcess@4 :proc
8   extern MessageBoxA@16 :proc
9
10  .data
11      wynik BYTE ?
12  .code
13
14  main proc
15
16          xor al,al;
17          mov al, 200;
18          add al, 100;   ≤4 ms upłynęło
19          mov wynik, al;
20          nop;
```

Rejestry

```
EAX = 00DBFE2C EBX = 01087000 ECX = 00E71005 EDX = 00E71005 ES

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 1
```

85 %

Pamięć 1 | Rejestry

Dezasemblacja | zad_if1.asm

```
1   .486
2   .model flat, stdcall
3   .stack 4096
4
5   ExitProcess PROTO, dwExitCode:DWORD
6
7   ;extern ExitProcess@4 :proc
8   extern MessageBoxA@16 :proc
9
10  .data
11      wynik BYTE ?
12  .code
13
14  main proc
15
16          xor al,al;
17          mov al, 200;
18          add al, 100;
19          mov wynik, al;
20          nop;
```

$2C_{16} \rightarrow 44_{10}$

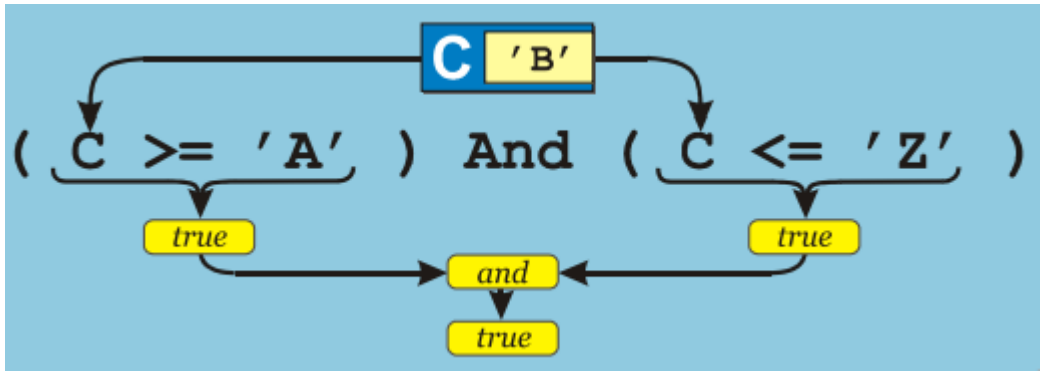
Dane wyjściowe

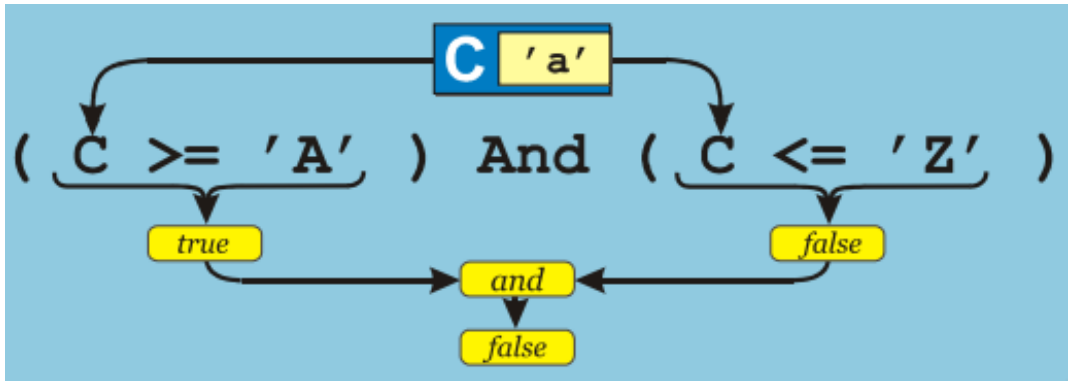
Pokaż dane wyjściowe z: Debugowanie

```
„Project1.exe” (Win32): załadowano „C:\Windows\SysWOW64\imm32.dll”.
Wątek 0x61d0 zakończył działanie z kodem 44 (0x2c).
Wątek 0x884c zakończył działanie z kodem 44 (0x2c).
Wątek 0x7088 zakończył działanie z kodem 44 (0x2c).
Program „[44256] Project1.exe” zakończył działanie z kodem 44 (0x2c).
```

Instrukcja warunkowa - warunek

W wyrażeniach logicznych mogą występować różne operatory logiczne **AND**, **OR** lub **NOT**





Instrukcja logiczna

operator logiczny **AND**

Operator logiczny And		
P	Q	P And Q
False	False	False
False	True	False
True	False	False
True	True	True

działanie: wykonuje operację **AND** (**koniunkcja**) na odpowiadających sobie bitach operandów **A** i **B**; wynik jest przechowywany w **A** (poprzednia wartość zostaje zamazana).

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość**, **rejestr** lub **adres pamięci** (ale tylko jeśli A również nie jest adresem pamięci).
modyfikowane flagi: OF i **CF** są zerowane; **SF**, ZF i **PF** nabywają wartość zależną od wyniku.

przykład: and edx, ebx

Instrukcja logiczna

operator negacji **NEG**

działanie: wykonuje działanie 0-A (odwraca znak liczby) i wynik przechowuje w **A**. Inna nazwa tej operacji to *uzupełnienie dwójkowe*. Polega to na zanegowaniu danej liczby i dodaniu jeden. Odwrócenie procesu polega na wykonaniu tej samej czynności

A: rejestr lub **adres pamięci**.

```
17    mov al, 5;
18    neg al;   ≤2 ms upłynęło
19    mov wynik, al;
```

```
Rejestry
EAX = 006FFF05 EBX = 005DA000 ECX = 00D91005 EDX = 00D91
  EIP = 00D91012 ESP = 006FFF70 EBP = 006FFF7C EFL = 00

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

przykład:

```
neg edx
neg al
```

```
17    mov al, 5;
18    neg al;
19    mov wynik, al;
```

```
Rejestry
EAX = 006FFFFB EBX = 005DA000 ECX = 00D91005 EDX = 00D91005 ESI
  EIP = 00D91014 ESP = 006FFF70 EBP = 006FFF7C EFL = 00000293

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 0 CY = 1
```

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
250	1	1	1	1	1	0	1	0
								+1
251	1	1	1	1	1	0	1	1

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
wynik	251 'ú'

Instrukcja logiczna

operator logiczny **NOT**

działanie: neguje (odwraca) wartości wszystkich bitów w A (1->0; 0->1).
W literaturze instrukcja ta zwana jest także *uzupełnieniem jedynkowym*

A: rejestr lub adres pamięci.

przykłady:

not edx

not al

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
250	1	1	1	1	1	0	1	0

```
17  mov al, 5;
18  not al;   ≤ 2 ms upłynęło
19  mov wynik, al;
```

```
17  mov al, 5;
18  not al;
19  mov wynik, al;
```

Rejestry

```
EAX = 010FFC05 EBX = 00E8C000 ECX = 00511005 EDX = 00511005 ESI
   EIP = 00511012 ESP = 010FFC24 EBP = 010FFC30 EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 010FFCFA EBX = 00E8C000 ECX = 00511005 EDX = 00511005 ESI = 00511005 EDI = 00511005
   EIP = 00511014 ESP = 010FFC24 EBP = 010FFC30 EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
wynik	250 'ú'
Dodaj element do oglądania	

Instrukcja logiczna

operator logiczny **AND**

Operator logiczny And		
P	Q	P And Q
False	False	False
False	True	False
True	False	False
True	True	True

działanie: wykonuje operację **AND** (**koniunkcja**) na odpowiadających sobie bitach operandów **A** i **B**; wynik jest przechowywany w **A** (poprzednia wartość zostaje zamazana).

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość**, **rejestr** lub **adres pamięci** (ale tylko jeśli A również nie jest adresem pamięci).
modyfikowane flagi: **OF** i **CF** są zerowane; **SF**, **ZF** i **PF** nabywają wartość zależną od wyniku.

przykład: and edx, ebx

Instrukcja logiczna

operator logiczny **AND**

przykład: and al, bl

Operator logiczny And		
P	*Q*	*P And Q*
False	False	False
False	True	False
True	False	False
True	True	True

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
249	1	1	1	1	1	0	0	1
1	0	0	0	0	0	0	0	1

```
17  mov al, 5;
18  mov bl,249;
19  and al,bl;
20  mov wynik, al;
```

Rejestry

```
EAX = 00BFFA05 EBX = 00D020F9 ECX = 00781005 EDX = 00781005 ESI
EIP = 00781014 ESP = 00BFFA60 EBP = 00BFFA6C EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17  mov al, 5;
18  mov bl,249;
19  and al,bl;
20  mov wynik, al;
```

Rejestry

```
EAX = 00BFFA01 EBX = 00D020F9 ECX = 00781005 EDX = 00781005 ESI
EIP = 00781016 ESP = 00BFFA60 EBP = 00BFFA6C EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 0
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
wynik	1 '\x1'

Dodaj element do oglądania

Instrukcja logiczna

operator logiczny **OR**

Operator logiczny Or		
P	Q	$P\ Or\ Q$
False	False	False
False	True	True
True	False	True
True	True	True

działanie: wykonuje operację **OR** (**alternatywa**) na odpowiadających sobie bitach operandów **A** i **B**; wynik jest przechowywany w **A** (poprzednia wartość zostaje zamazana).

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość**, **rejestr** lub **adres pamięci** (ale tylko jeśli **A** również nie jest adresem pamięci).
modyfikowane flagi: **OF** i **CF** są zerowane; **SF**, **ZF** i **PF** nabywają wartość zależną od wyniku.

przykład: or edx,ebx

Instrukcja logiczna

operator logiczny **OR**

przykład: or al, bl

Operator logiczny Or		
P	Q	P Or Q
False	False	False
False	True	True
True	False	True
True	True	True

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
249	1	1	1	1	1	0	0	1
253	1	1	1	1	1	1	0	1

```
17    mov al, 5;
18    mov bl,249;
19    or al,bl;
20    mov wynik, al;
```

Rejestry

```
EAX = 006FFC05 EBX = 0055A0F9 ECX = 00E01005 EDX = 00E0100
   EIP = 00E01014 ESP = 006FFC1C EBP = 006FFC28 EFL = 0006

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17    mov al, 5;
18    mov bl,249;
19    or al,bl;
20    mov wynik, al;
```

Rejestry

```
EAX = 006FFCFD EBX = 0055A0F9 ECX = 00E01005 EDX = 00E01005 ESI
   EIP = 00E01016 ESP = 006FFC1C EBP = 006FFC28 EFL = 00000282

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 0 CY = 0
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
wynik	253 'ý'

Dodaj element do oglądania

Instrukcja logiczna

operator logiczny **XOR**

Input		Output
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

działanie: wykonuje operację **XOR** (alternatywa wykluczająca) na odpowiadających sobie bitach **A** i **B**; wynik jest przechowywany w **A** (poprzednia wartość zostaje zamazana).

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość**, **rejestr** lub **adres pamięci** (ale tylko jeśli A również nie jest adresem pamięci).
modyfikowane flagi: OF i **CF** są zerowane; **SF**, **ZF** i **PF** nabywają wartość zależną od wyniku.

przykład: xor edx, ebx

Instrukcja logiczna

operator logiczny **XOR**

przykład: xor al bl

Input		Output
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
249	1	1	1	1	1	0	0	1
252	1	1	1	1	1	1	0	0

```
17    mov al, 5;
18    mov bl,249;
19    xor al,bl;
20    mov wynik, al;
```

```
Rejestry
EAX = 00B3FB05 EBX = 009EE0F9 ECX = 01001005 EDX = 01001005 E
   EIP = 01001014 ESP = 00B3FAC4 EBP = 00B3FAD0 EFL = 000002
OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17    mov al, 5;
18    mov bl,249;
19    xor al,bl;
20    mov wynik, al;
```

```
Rejestry
EAX = 00B3FBFC EBX = 009EE0F9 ECX = 01001005 EDX = 01001005 ESI =
   EIP = 01001016 ESP = 00B3FAC4 EBP = 00B3FAD0 EFL = 00000286
OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 1 CY = 0
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
wynik	252 'ü'

Instrukcja logiczna

operator logiczny **TEST**

działanie: wykonuje operację **AND** (**koniunkcja**) na odpowiadających sobie bitach operandów **A** i **B**; działa podobnie jak instrukcja and, jednak nie zapisuje nigdzie wyniku operacji.
Ustawia jedynie stosowne flagi, w sposób identyczny w jaki robi to instrukcja and.

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość**, **rejestr** lub **adres pamięci** (ale tylko jeśli A również nie jest adresem pamięci).
modyfikowane flagi: **OF** i **CF** są zerowane; **SF**, **ZF** i **PF** nabywają wartość zależną od wyniku; stan **AF** staje się niezdefiniowany.

przykład: test edx, ebx

Instrukcja logiczna

operator logiczny **TEST**

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
249	1	1	1	1	1	0	0	1
1	0	0	0	0	0	0	0	1

```
17  mov al, 5;
18  mov bl,249;
19  test al,bl;
20  mov wynik, al;
```

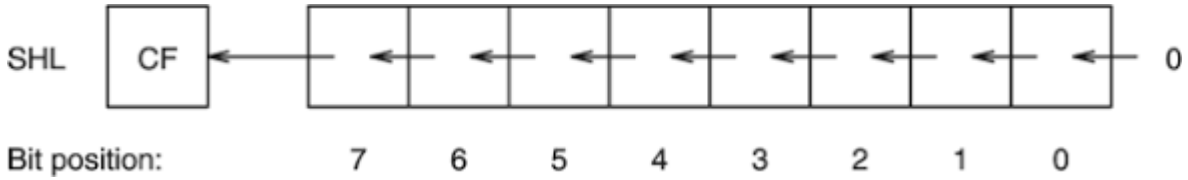
```
Rejestry
EAX = 00AFFF05 EBX = 008CF0F9 ECX = 00DF1005 EDX = 00DF1005 ESI = 00DF
   EIP = 00DF1014 ESP = 00AFFEDC EBP = 00AFFEE8 EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17  mov al, 5;
18  mov bl,249;
19  test al,bl;
20  mov wynik, al;
```

```
Rejestry
EAX = 00AFFF05 EBX = 008CF0F9 ECX = 00DF1005 EDX = 00DF1005 ESI =
   EIP = 00DF1016 ESP = 00AFFEDC EBP = 00AFFEE8 EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 0
```

przykład: test al, bl

Instrukcja logiczna



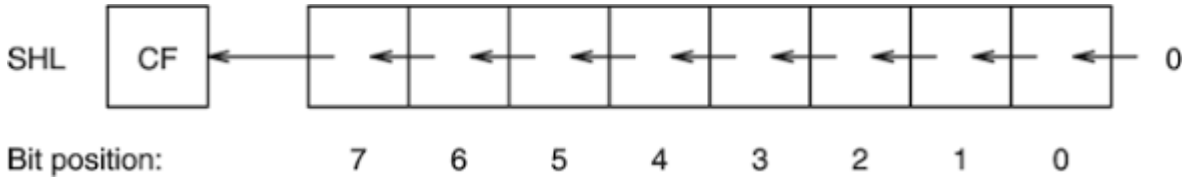
operator logiczny **SHL – przesunięcie logiczne w lewo**

działanie: przesuwa wszystkie bity **A** w lewo o ilość bitów zdefiniowaną przez **B**

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość** lub **rejestr CL** (8-bitowa część rejestru ECX).
modyfikowane flagi: **CF** (przyjmuje wartość ostatniego bitu "wyrzuconego" poza obręb); **SF**, **ZF** i **PF** (zależnie od wyniku), **OF**(tylko, gdy B=1; przyjmuje wartość 0, gdy dwa ostatnie bity przed przesunięciem były równe; przyjmuje 1 w przeciwnym przypadku).

przykład: shl edx, 02h

Instrukcja logiczna



operator logiczny **SHL – przesunięcie logiczne w lewo**

	7	6	5	4	3	2	1	0
5	0	0	0	0	0	1	0	1
20	0	0	0	1	0	1	0	0

```
17  mov al, 5;
18  shl al,2;
19  mov wynik, al;
```

Rejestry

```
EAX = 012FFF05 EBX = 010AE000 ECX = 00871005 EDX = 00871005 E
   EIP = 00871012 ESP = 012FFF6C EBP = 012FFF78 EFL = 0000024

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17  mov al, 5;
18  shl al,2;
19  mov wynik, al;
```

Rejestry

```
EAX = 012FFF14 EBX = 010AE000 ECX = 00871005 EDX = 00871005 ESI
   EIP = 00871015 ESP = 012FFF6C EBP = 012FFF78 EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

	7	6	5	4	3	2	1	0
200	1	1	0	0	1	0	0	0
32	0	0	1	0	0	0	0	0

```
17  mov al, 200;
18  shl al,2;
19  mov wynik, al;
```

Rejestry

```
EAX = 0073FCC8 EBX = 005A9000 ECX = 00061005 EDX = 00061005 ES
   EIP = 00061012 ESP = 0073FC40 EBP = 0073FC4C EFL = 0000024
OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
17  mov al, 200;
18  shl al,2;
19  mov wynik, al;
```

Rejestry

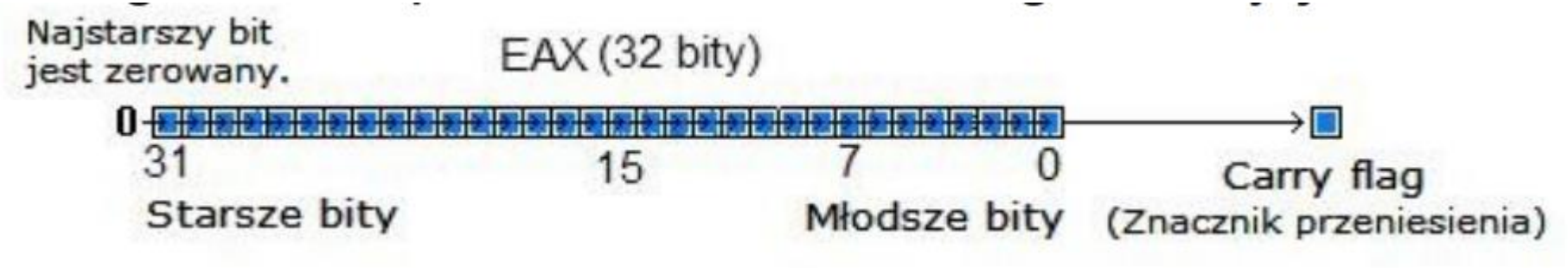
```
EAX = 0073FC20 EBX = 005A9000 ECX = 00061005 EDX = 00061005 ESI
   EIP = 00061015 ESP = 0073FC40 EBP = 0073FC4C EFL = 00000203

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 1
```

Instrukcja logiczna

operator logiczny **SHR – przesunięcie logiczne w prawo**

działanie: przesuwa wszystkie bity **A** w prawo o ilość bitów zdefiniowaną przez **B**

A: **rejestr** lub **adres pamięci**.
B: **konkretna wartość** lub **rejestr CL** (8-bitowa część rejestru ECX).
modyfikowane flagi: **CF** (przyjmuje wartość ostatniego bitu "wyrzuconego" poza obręb); **SF**, **ZF** i **PF** (zależnie od wyniku), **OF**(tylko, gdy B=1; przyjmuje wartość ostatniego bitu A przed przesunięciem).



przykład: shr edx, 02h

Instrukcja logiczna

operator logiczny **SHR – przesunięcie logiczne w prawo**

```
xor eax,eax
mov ax,48
shr ax,2
```

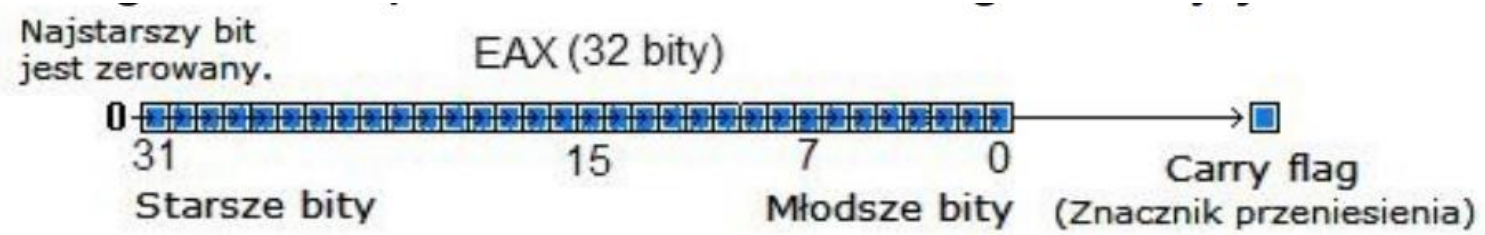
```
Rejestry
EAX = 00000030 EBX = 0077F000 ECX = 00161005
  EDX = 00161005 ESI = 00161005 EDI = 00161005
  EIP = 00161016 ESP = 0095FF08 EBP = 0095FF14
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 0000000C EBX = 0077F000 ECX = 00161005
  EDX = 00161005 ESI = 00161005 EDI = 00161005
  EIP = 0016101A ESP = 0095FF08 EBP = 0095FF14
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

```
xor eax,eax
mov ax,5
shr ax,1
```

```
Rejestry
EAX = 00000005 EBX = 008FA000 ECX = 007B1005
  EDX = 007B1005 ESI = 007B1005 EDI = 007B1005
  EIP = 007B1016 ESP = 0065FE64 EBP = 0065FE70
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 00000002 EBX = 008FA000 ECX = 007B1005
  EDX = 007B1005 ESI = 007B1005 EDI = 007B1005
  EIP = 007B1019 ESP = 0065FE64 EBP = 0065FE70
  EFL = 00000203

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 1
```



Instrukcje skoku – asembler

- Skok bezwarunkowy JMP
- Skoki warunkowe JZ,JNZ,…

(wykonywane gdy spełniony jest określony warunek)

JZ JE	skok, jeśli zero skok, jeśli równe	ZF=1
JNZ JNE	skok, jeśli nie zero skok, jeśli nie równe	ZF=0
JA JNBE	skok, jeśli większe (bez znaku) skok, jeśli nie mniejsze lub równe (bez znaku)	CF=0 i ZF=0
JNA JBE	skok, jeśli nie większe (bez znaku) skok, jeśli mniejsze lub równe (bez znaku)	CF=1 lub ZF=1
JB JNAE JC	skok, jeśli mniejsze (bez znaku) skok, jeśli nie większe lub równe (bez znaku) skok, jeśli jest przeniesienie	CF=1
JNB JAE JNC	skok, jeśli nie mniejsze (bez znaku) skok, jeśli większe lub równe (bez znaku) skok, jeśli nie ma przeniesienia	CF=0
JG JNLE	skok, jeśli większe (ze znakiem) skok, jeśli nie mniejsze lub równe (ze znakiem)	ZF=0 i SF=OF

JNG JLE	skok, jeśli nie większe (ze znakiem) skok, jeśli mniejsze lub równe (ze znakiem)	ZF=1 lub SF<>OF
JL JNGE	skok, jeśli mniejsze (ze znakiem) skok, jeśli nie większe lub równe (ze znakiem)	SF<>OF
JNL JGE	skok, jeśli nie mniejsze (ze znakiem) skok, jeśli większe lub równe (ze znakiem)	SF=OF
JCXZ	skok, jeśli rejestr CX=0	CX=0
JECXZ	skok, jeśli rejestr ECX=0	ECX=0
JP, JPE	skok, jeśli parzystość	PF=1
JNP, JPO	skok, jeśli nieparzystość	PF=0
JO	skok, jeśli przepełnienie	OF=1
JNO	skok, jeśli nie ma przepełnienia	OF=0
JS	skok, jeśli znak (ujemny)	SF=1
JNS	skok, jeśli nie ma znaku (dodatni)	SF=0

- Wywołanie procedury/funkcji CALL
- Powrót z procedury/funkcji RET

ang. return from interrupt

Instrukcje skoku – asembler

warunek		działanie	negacja warunku	
je (lub jz)	=	jump if equal	jne (lub jnz)	! =
bez znaku				
ja	>	jump if above	jna	! >
jae	>=	jump if above or equal	jnae	! >=
jb	<	jump if below	jnb	! <
jbe	<=	jump if below or equal	jnbe	! <=
ze znakiem				
jg	>	jump if greater	jng	! >
jge	>=	jump if greater or equal	jnge	! >=
jl	<	jump if less	jnl	! <
jle	<=	jump if less or equal	jnle	! <=

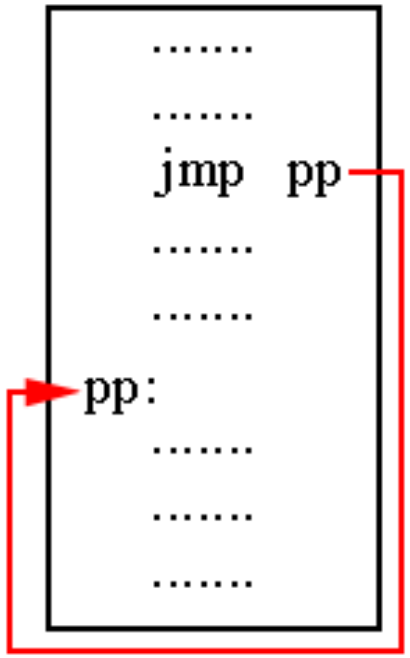
Instrukcje skoku – JMP

jmp etykieta

polecenie wykonuje bezwarunkowy skok do etykiety lub adresu.

Etykieta może być typu

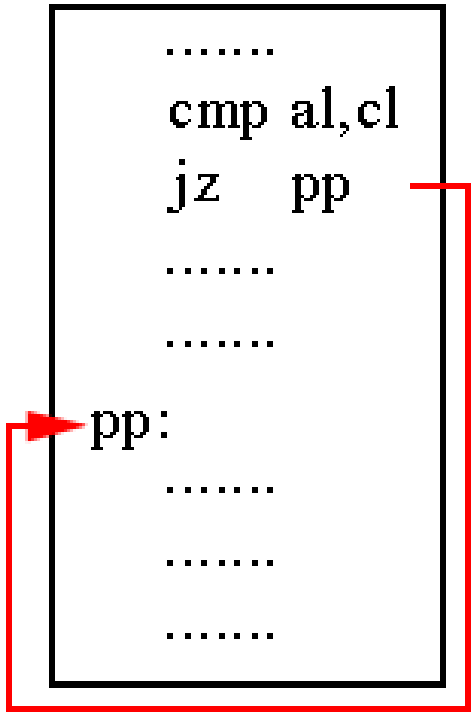
short (-128 lub 127 bajtów od aktualnego położenia),
near (aktualny segment) lub
far (inny segment).



Instrukcje skoku warunkowego

jz etykieta Jump if Zero

polecenie wykonuje skok do etykiety lub adresu jeżeli flaga **ZF**=1.



Instrukcja IF

Debug | x86 | Lokalny debuger Windows

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax, 5    ;
        mov ebx,4     ;
        cmp eax,5     ; porównanie EAX z 5
        je et1        ; jeśli równe to skok
        add ebx,2     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

Instrukcja IF

Debug | x86 | Lokalny debuger Windows

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax, 5    ;
        mov ebx,4     ;
        cmp eax,5     ; porównanie EAX z 5
        je et1        ; jeśli równe to skok
        add ebx,2     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

Rejestry

```
EAX = 00000005 EBX = 00000004 ECX = 00FB1005 EDX = 00FB1005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000005 EBX = 00000004 ECX = 00FB1005 EDX = 00FB1

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
main proc

        mov eax, 5    ;
        mov ebx,4     ;
        cmp eax,5     ; porównanie EAX
        je et1        ; jeśli równe to
        add ebx,2     ;
 et1:
        INVOKE ExitProcess, ebx   ≤ 1 ms upłynęło
```

Rejestry

```
EAX = 00000005 EBX = 00000004 ECX = 00FB1005 EDX = 00FB1

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Konsola debugowania progra

```
C:\vs2019_c\arch_komp\if\pr2\Project1\Debug\Project1.exe (proces 3952) zakończono z kodem 4.
Aby automatycznie zamknąć konsole po zatrzymaniu debugowania, włącz opcje Narzędzia -> Opcje
```

Instrukcja IF

```
10  main proc
11
12          mov eax,3     ;
13          mov ebx,4     ;
14          cmp eax,5     ; porównanie EAX z 5
15          je et1        ; jeśli równe to skok
16          add ebx,2     ;
17   et1:
18          INVOKE ExitProcess, ebx
19          nop;
20  main endp
```

Rejestry

```
EAX = 00000003 EBX = 00000004 ECX = 00CD1005 EDX = 00CD1005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000003 EBX = 00000004 ECX = 00CD1005 EDX = 00CD1005

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 0 CY = 1
```

```
14          cmp eax,5     ; porównanie EAX z 5
15          je et1        ; jeśli równe to skok
16          add ebx,2     ;  ≤ 1 ms upłynęło
17   et1:
```

Rejestry

```
EAX = 00000003 EBX = 00000006 ECX = 00CD1005 EDX = 00CD1005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

Konsola debugowania progra

```
C:\vs2019_c\arch_komp\if\pr2\Project1\Debug\Project1.exe (proces 21508) zakończono z kodem 6.
```

Instrukcja IF – złożony warunek logiczny

Debug | x86 | Lokalny debuger Windows | Auto

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax,5     ; czy jest w przedziale <2,6>
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

Jeżeli **EAX** należy do przedziału <2,6>
to rejestr **EBX** ma wartość 1
w przeciwnym wypadku wartość **EBX** równa się 0

Instrukcja IF – złożony warunek logiczny

Debug | x86 | Lokalny debuger Windows | Auto

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax,5     ; czy jest w przedziale <2,6>
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

JZ JE	skok, jeśli zero skok, jeśli równe	ZF=1
JNZ JNE	skok, jeśli nie zero skok, jeśli nie równe	ZF=0
JA JNBE	skok, jeśli większe (bez znaku) skok, jeśli nie mniejsze lub równe (bez znaku)	CF=0 i ZF=0
JNA JBE	skok, jeśli nie większe (bez znaku) skok, jeśli mniejsze lub równe (bez znaku)	CF=1 lub ZF=1
JB JNAE JC	skok, jeśli mniejsze (bez znaku) skok, jeśli nie większe lub równe (bez znaku) skok, jeśli jest przeniesienie	CF=1
JNB JAE JNC	skok, jeśli nie mniejsze (bez znaku) skok, jeśli większe lub równe (bez znaku) skok, jeśli nie ma przeniesienia	CF=0
JG JNLE	skok, jeśli większe (ze znakiem) skok, jeśli nie mniejsze lub równe (ze znakiem)	ZF=0 i SF=OF

Instrukcja IF – złożony warunek logiczny

Debug | x86 | Lokalny debuger Windows | Auto

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax,5     ; czy jest w przedziale <2,6>
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

EAX=5

Rejestry

EAX = 00000005 EBX = 00000000 ECX = 00541005 EDX = 00541005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0

Rejestry

EAX = 00000005 EBX = 00000000 ECX = 00541005 EDX = 00541005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0

```
14  cmp eax,2     ;
15  jb et1        ; jeśli < od 2 to skok
16  cmp eax,6     ; ≤ 1 ms upłynęło
```

Rejestry

EAX = 00000005 EBX = 00000000 ECX = 00541005 EDX = 00541005

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 1 CY = 1

```
17  ja et1        ; jeżeli > od 6 to skok
18  add ebx,1     ; ≤ 1 ms upłynęło
```

Rejestry

EAX = 00000005 EBX = 00000001 ECX = 00541005 EDX = 00541005

Konsola debugowania progra

```
C:\vs2019_c\arch_komp\if\pr3\Project1\Debug\Project1.exe (proces 25324) zakończono z kodem 1.
Aby automatycznie zamknąć konsolę po zatrzymaniu debugowania, włącz opcję Narzędzia -> Opcje ->
```

Instrukcja IF – złożony warunek logiczny

Debug | x86 | Lokalny debuger Windows | Auto

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax,5     ; czy jest w przedziale <2,6>
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

Rejestry

```
EAX = 00000001 EBX = 00000000 ECX = 009A1005 EDX = 009A1005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000001 EBX = 00000000 ECX = 009A1005 EDX = 009A1005

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 1 CY = 1
```

```
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
et1:
        INVOKE ExitProcess, ebx   ≤ 1 ms upłynęło
```

EAX=1

- CY - CF (Carry Flag)
- PE - PF (Parity Flag)
- AC - AF (Auxiliary Carry Flag)
- ZR - ZF (Zero Flag)
- PL - SF (Sign Flag)
- EI - IF (Interrupt Enable Flag)
- UP - DF (Direction Flag)
- OV - OF (Overflow Flag)

JZ JE	skok, jeśli zero skok, jeśli równe	ZF=1
JNZ JNE	skok, jeśli nie zero skok, jeśli nie równe	ZF=0
JA JNBE	skok, jeśli większe (bez znaku) skok, jeśli nie mniejsze lub równe (bez znaku)	CF=0 i ZF=0
JNA JBE	skok, jeśli nie większe (bez znaku) skok, jeśli mniejsze lub równe (bez znaku)	CF=1 lub ZF=1
JB JNAE JC	skok, jeśli mniejsze (bez znaku) skok, jeśli nie większe lub równe (bez znaku) skok, jeśli jest przeniesienie	CF=1
JNB JAE	skok, jeśli nie mniejsze (bez znaku) skok, jeśli większe lub równe (bez znaku)	CF=0

Instrukcja IF – złożony warunek logiczny

Debug | x86 | Lokalny debuger Windows | Auto

main.asm

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD
.data

.code

main proc

        mov eax,5     ; czy jest w przedziale <2,6>
        mov ebx,0     ;
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
 et1:
        INVOKE ExitProcess, ebx
        nop;
main endp
end
```

Rejestry

```
EAX = 00000001 EBX = 00000000 ECX = 009A1005 EDX = 009A1005

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000001 EBX = 00000000 ECX = 009A1005 EDX = 009A1005

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 1 CY = 1
```

```
        cmp eax,2     ;
        jb et1        ; jeśli < od 2 to skok
        cmp eax,6     ;
        ja et1        ; jeżeli > od 6 to skok
        add ebx,1     ;
et1:
        INVOKE ExitProcess, ebx   ≤ 1 ms upłynęło
```

EAX=1

Konsola debugowania progra

```
C:\vs2019_c\arch_komp\if\pr3\Project1\Debug\Project1.exe (proces 27028) zakończono z kodem 0.
```

Pętle programowe – asembler – pętla FOR

Instrukcja skoku jmp

Debug | x86 | Lokalny debug

ConsoleApplication1.cpp

ConsoleApplication1 | (Globalny z

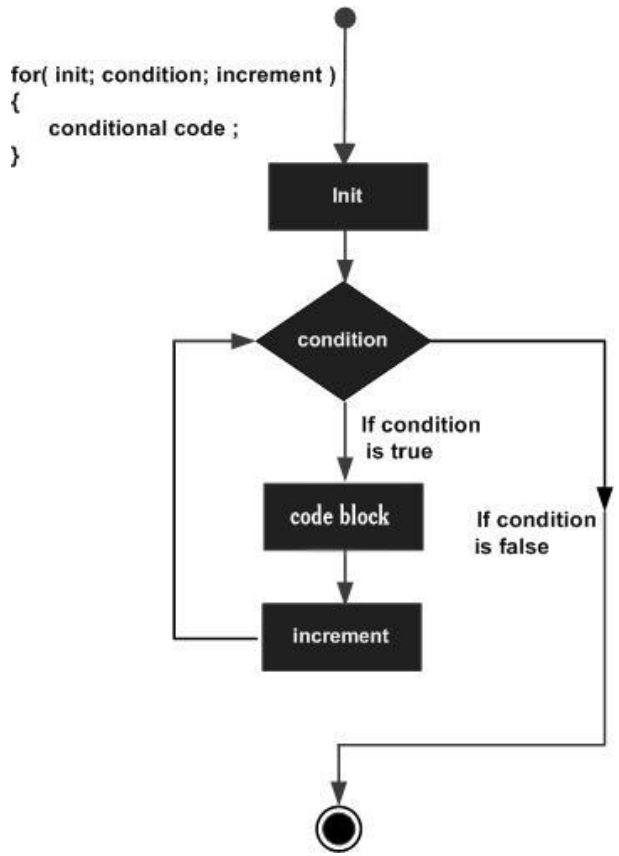
```cpp
#include <iostream>

int main()
{
    short i, j,k;
    i = 7; j = 9;
    __asm mov ax, 0;
    __asm mov cx, j;
    __asm start:
    __asm cmp cx, 0;
    __asm je koniec;
    __asm add ax, i;
    __asm sub cx, 1;
    __asm jmp start;
    __asm koniec:
    __asm mov k, ax;


    std::cout << k << " \n";
}
```

Konsola debugowania progra

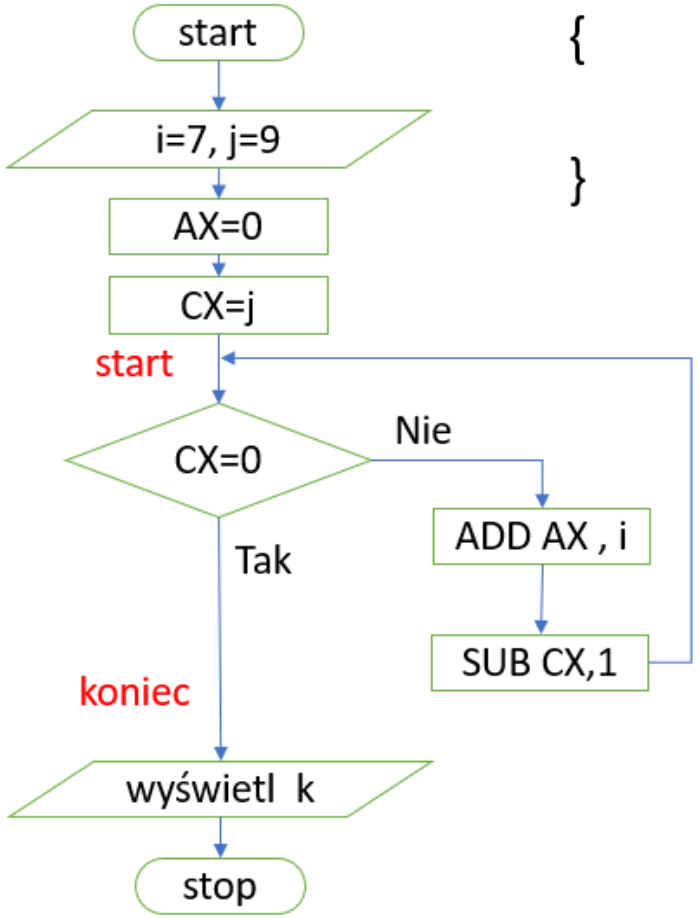
```
63

C:\vs2022_c\arch_komp\petla_for\pr1\ConsoleApplic
1620158712.
Aby automatycznie zamknąć konsolę po zatrzymaniu
znie zamknij konsolę po zatrzymaniu debugowania.
Naciśnij dowolny klawisz, aby zamknąć to okno...
```

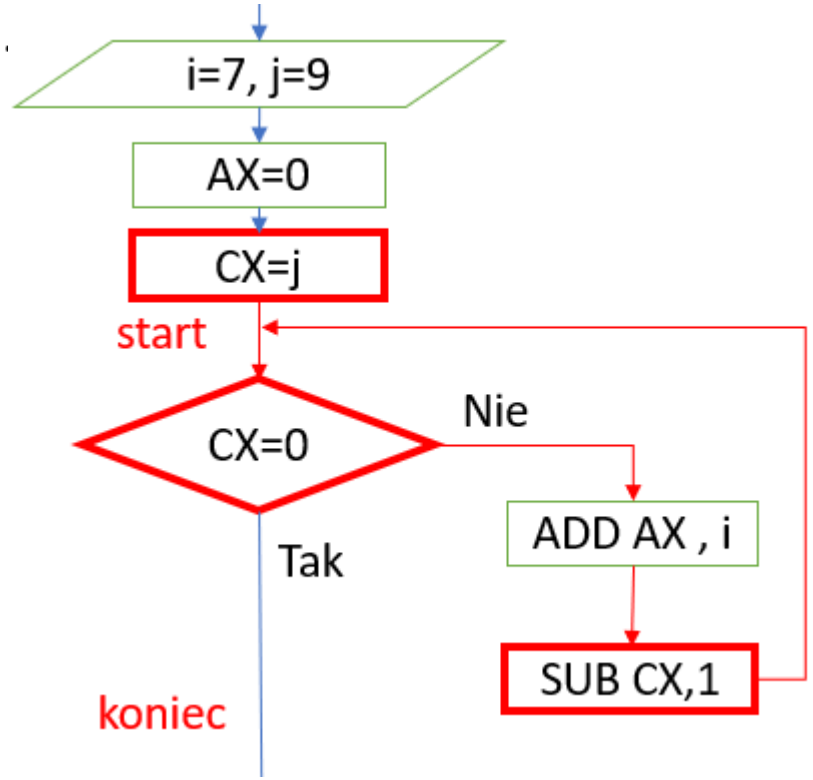
Pętle programowe – asembler – pętla FOR

Instrukcja skoku jmp




```
for(int j=9;j>0;j--)
{

}
```





Pętle programowe – asembler – pętla FOR

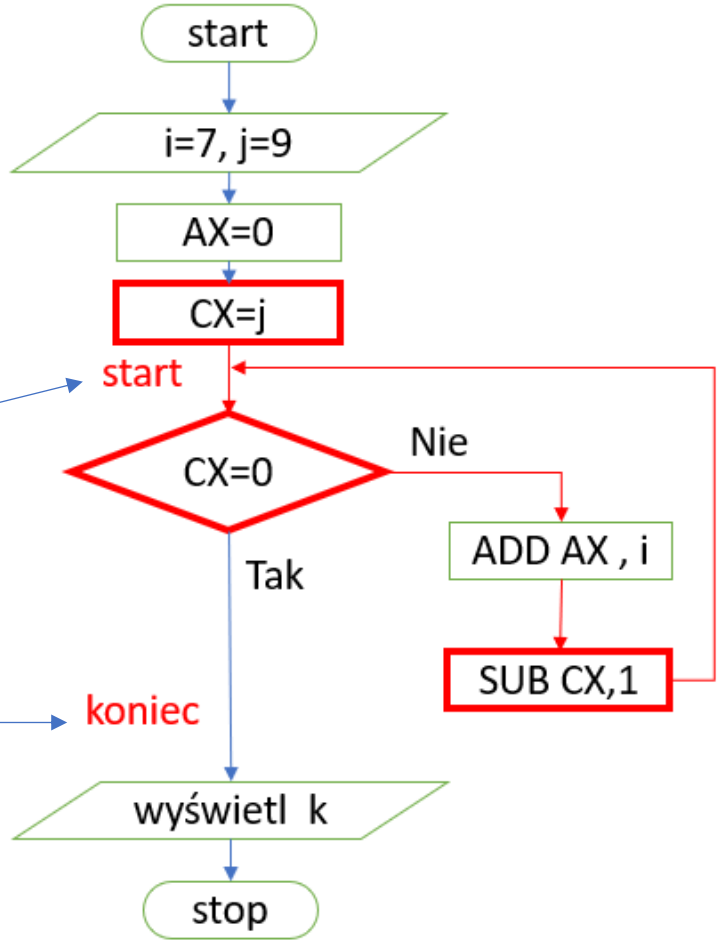
Instrukcja skoku jmp

Debug | x86 | Lokalny debug

ConsoleApplication1.cpp

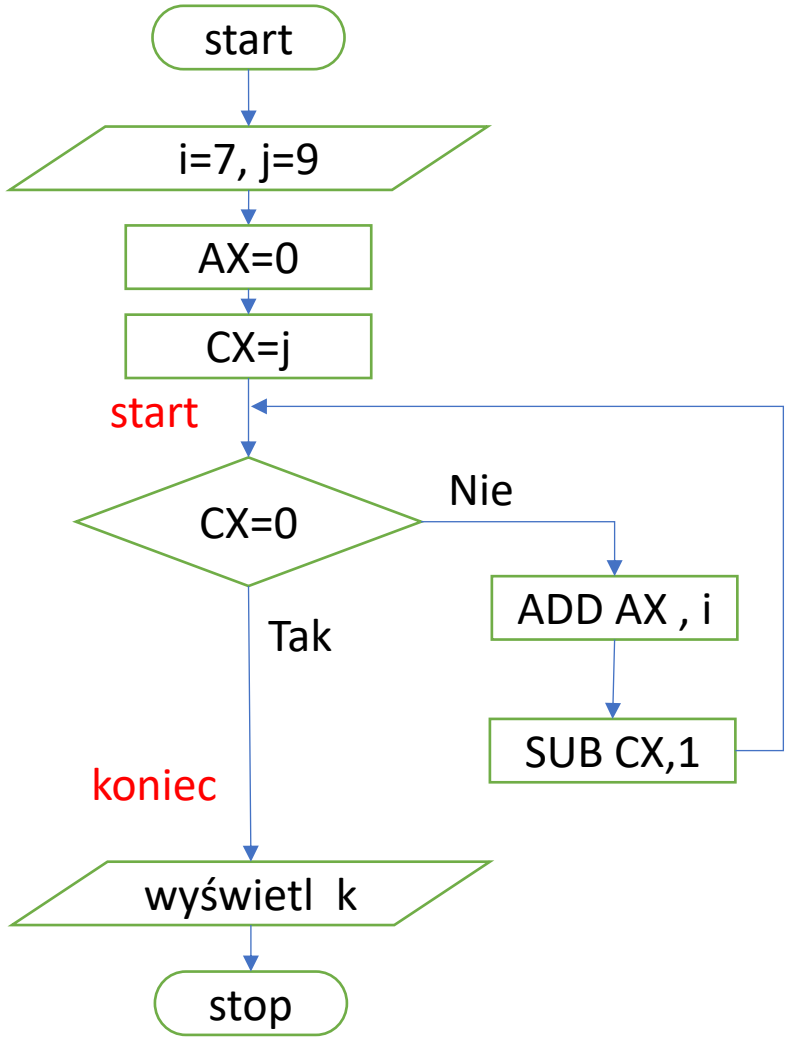
ConsoleApplication1 | (Globalny z

```
1   #include <iostream>
2
3   int main()
4   {
5       short i, j,k;
6       i = 7; j = 9;
7       __asm mov ax, 0;
8       __asm mov cx, j;
9       __asm start:
10      __asm cmp cx, 0;
11      __asm je koniec;
12      __asm add ax, i;
13      __asm sub cx, 1;
14      __asm jmp start;
15      __asm koniec:
16      __asm mov k, ax;
17
18
19      std::cout << k << " \n";
20  }
21
```



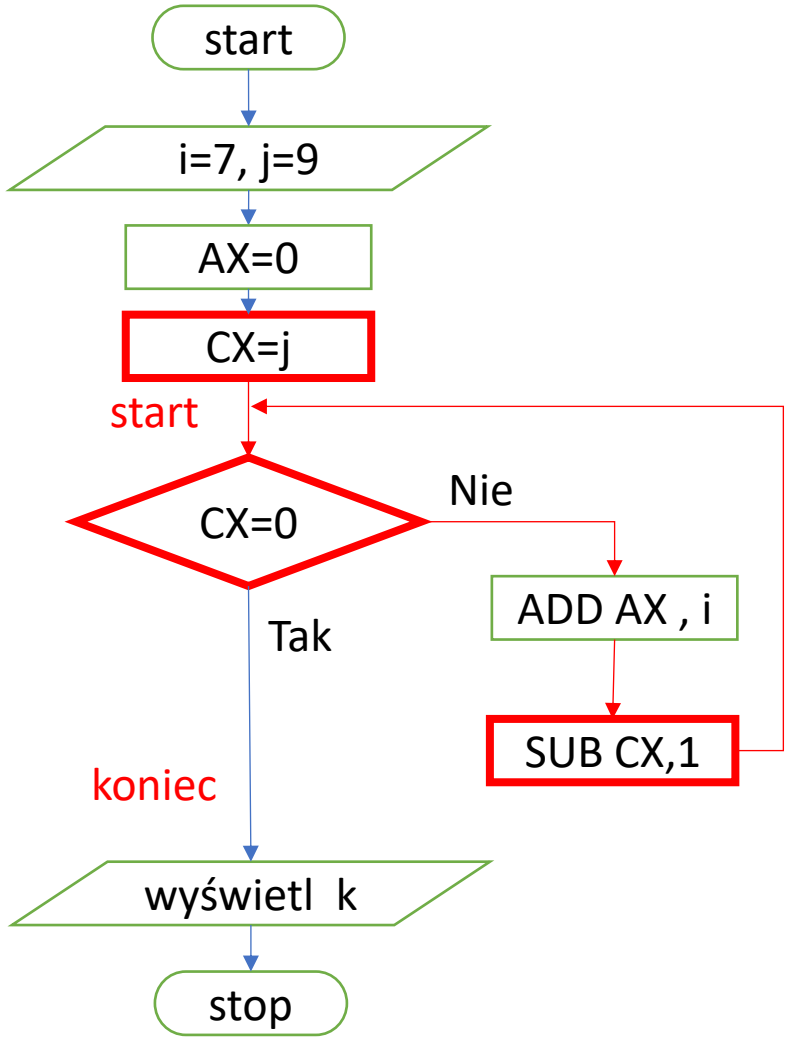
Pętle programowe – asembler

Pętla FOR



Pętle programowe – asembler

Pętla FOR



Pętle programowe – asembler

Instrukcja skoku LOOP

pętla; zmniejsza wartość rejestru CX o jeden i jeśli CX jest większe od 0, to powoduje bezwarunkowy przeskok do etykiety

loop etykieta

```
    mov eax, 0
    mov ecx, 10
poczatek_petli:
    add eax, ecx
    loop poczatek_petli
```

```
int suma=0;
for(int i=10; i!=0; --i)
  suma += i;
```

Pętle programowe – asembler

Instrukcja skoku LOOP

```
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD

.data
    suma dd ?

.code
main proc
    mov eax,0;
    mov ecx,10;
poczatek_petli:
    add eax,ecx;
    loop poczatek_petli;
    mov suma, eax;
    INVOKE ExitProcess, suma
main endp
end
```

```
int suma=0;
for(int i=10; i!=0; --i)
   suma += i;
```

suma=55

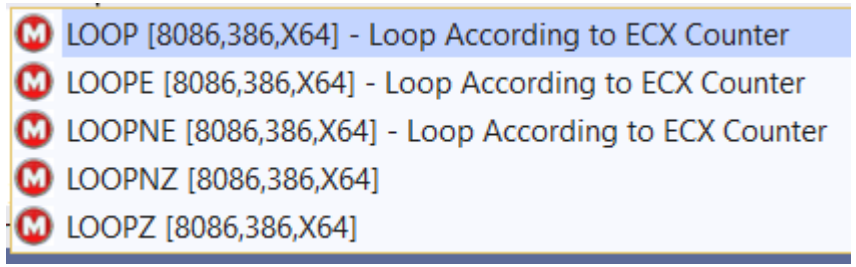
Konsola debugowania progra

```
C:\vs2019_c\arch_komp\instrukcje\loop\pr1\Project1\Debug\Project1.exe (proces 5200) zakończono z kodem 55.
Aby automatycznie zamknąć konsolę po zatrzymaniu debugowania, włącz opcję Narzędzia -> Opcje -> Debugowanie -> Automatyc
```

Pętle programowe – asembler

Instrukcja skoku – wariantowa LOOPZ, LOOPNZ

```
LOOPZ etykieta     ; wykonuje skok jeżeli ecx!=0 oraz ZF=1

LOOPNZ etykieta    ; wykonuje skok jeżeli ecx!=0 oraz ZF=0
```



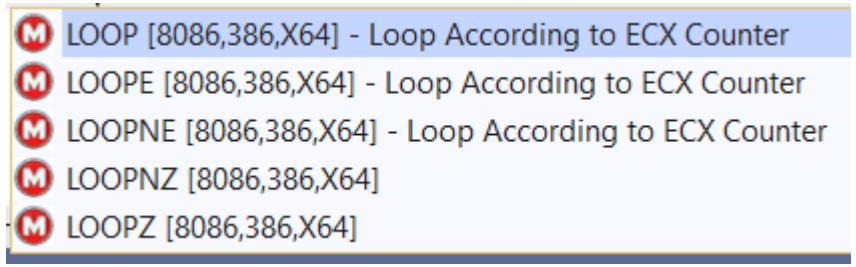
LOOPE, LOOPNE, LOOPZ, LOOPNZ działają następująco :

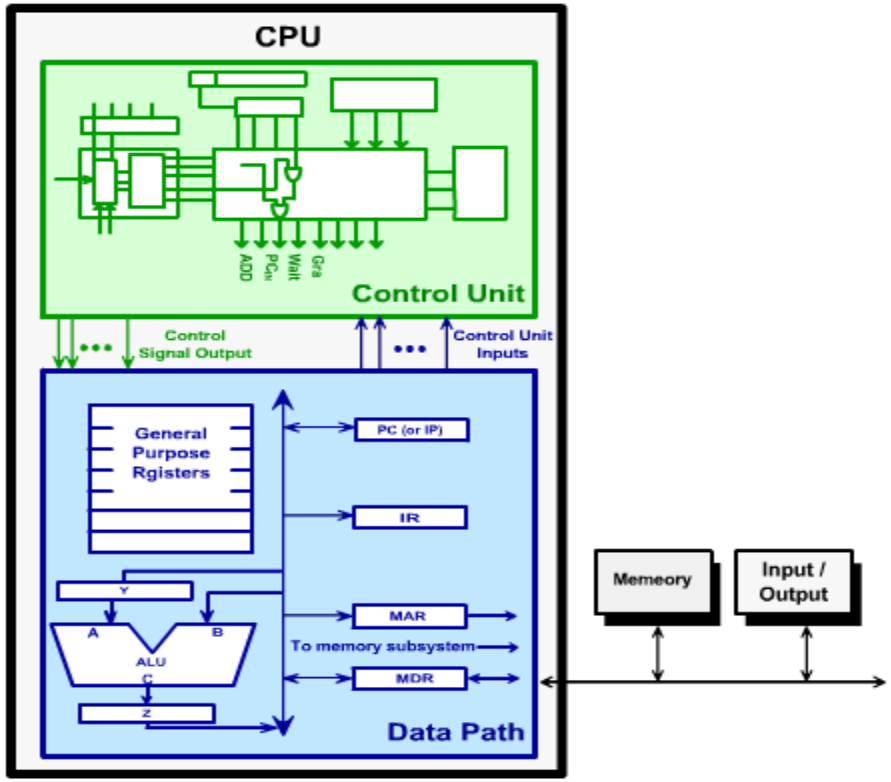
- zmniejsza CX o jeden (DEC CX - tak jak przy LOOP)
- jeśli CX>0 wykonuje skok warunkowy do etykiety w zależności od typu instrukcji: LOOPE - JE, LOOPNE - JNE, LOOPZ - JZ, LOOPNZ - JNZ.
- jeśli CX=0 lub nie spełniono warunku skoku warunkowego, to zakańcza pętlę

Pętle programowe – asembler

Instrukcja skoku – wariantowa LOOPZ, LOOPNZ

```
LOOPZ etykieta     ; wykonuje skok jeżeli ecx!=0 oraz ZF=1

LOOPNZ etykieta    ; wykonuje skok jeżeli ecx!=0 oraz ZF=0
```





Dziękuję za uwagę